UNIVERSIDAD DE BURGOS

Curso: 2025-2026

SISTEMAS DISTRIBUIDOS

PRÁCTICA 2 - Sistema de gestión de excepciones con Springboot y Python

Alumno: Alfonso Domingo Blanco

INDICE

1. Enunciado Práctica 2 y Entrega (enlace github) 3

2. Capturas 4
    a. docker-compose.yml 4
    b. pom.xml 5
    c. application.properties 6
    d. .env 6
    e. Código HTML 7
    f. app.py 10

3. Prueba de Ejecución 11
    a. Capturas de los servidores en funcionamiento 11
    b. Página de Inicio 12
    c. Página de Login 13
    d. Página de Test 14

4. Estado de las operaciones en los servidores 16

1. Enunciado Práctica 2

El objetivo de la Práctica Obligatoria 2 poner en marcha un sistema de gestion de excepciones con springboot y pythonI.

Sistema con spring boot Frontend y api en python

El objetivo de la práctica es crear y desarrollar un Frontend , un backend y un API con Flask.

La aplicación constará al menos de:

1. Una página principal
2. Una pantalla de login sencilla
3. Una pantalla para simular invocaciones a un API de terceros (Python)
4. Un API realizada en Python que acceda a una base de datos.

Las tecnologías sugeridas son, Spring Boot , JPA , Hibernate , y Thymeleaf.

El enfoque se centra en la implementación de un sistema que sea capaz de mostrar cómo se tratan las diferentes excepciones tanto a nivel de acceso a datos como a nivel de llamadas a apis de terceros.

El conjunto de apis desarrollados en Python debe de permitir hacer todos los tipos de test disponibles sobre excepciones, a modo de ejemplo:

1. Excepciones de apretura y lectura de archivos.
2. Excepciones de accesos a bases de datos
3. Excepciones de llamadas a APIs de terceros (pokemon)

También se deben simular usando postman errores.

Se tratarán excepciones de modo que en caso de no ser críticas salga en el front la información traducida del error.

ENTREGA: https://github.com/fonsydb/P2_Excepciones

2. Capturas

a. docker-compose.yml

docker-compose.yml

```yaml
services:
  mysqldb:
    image: mysql:9.6.0
    restart: unless-stopped
    environment:
      - MYSQL_ROOT_PASSWORD=$MYSQLDB_ROOT_PASSWORD
      - MYSQL_DATABASE=$MYSQLDB_DATABASE
    ports:
      - $MYSQLDB_LOCAL_PORT:$MYSQLDB_DOCKER_PORT
    volumes:
      - db:/var/lib/mysql
  postgres:
    image: postgres:15
    container_name: postgres-db
    restart: always
    env_file: ./.env
    environment:
      POSTGRES_DB: $POSTGRES_DB
      POSTGRES_USER: $POSTGRES_USER
      POSTGRES_PASSWORD: $POSTGRES_PASSWORD
    ports:
      - $POSTGRES_PORT:5432
    volumes:
      - postgres_data:/var/lib/postgresql/data

  volumes:
    postgres_data:
volumes:
    db:
```

b. pom.xml

docker-compose.yml | pom.xml (basico)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://
    <modelVersion>4.0.0</modelVersion>
    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>4.0.3</version> <!--4.0.3-->
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.sistemasdistr</groupId>
    <artifactId>basico</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>basico</name>
    <description>Demo project for Spring Boot</description>
    <url/>
    <licenses>
        <license/>
    </licenses>
    <developers>
        <developer/>
    </developers>
    <scm>
        <connection/>
        <developerConnection/>
        <tag/>
        <url/>
    </scm>
    <properties>
        <java.version>21</java.version>
    </properties>
    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
```

Compatible with Maven 3

12

c. **application.properties**

```
spring.application.name=basicosd
#Indica el driver/lib para conectar java a mysql
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

#Url donde esta el servicio de tu mysql y el nombre de la base de datos
spring.datasource.url=jdbc:mysql://localhost:3806/basicosd

#Usuario y contrasena para tu base de datos descrita en la linea anterio
spring.datasource.username=root
spring.datasource.password=eneas2805

#[opcional]Imprime en tu consola las instrucciones hechas en tu base de
spring.jpa.show-sql = true
spring.jpa.hibernate.ddl-auto=create-drop
#habilitamos la pagina de error generico
server.port=8099
```

d. .env

```
MYSQLDB_USER=root
MYSQLDB_ROOT_PASSWORD=eneas2805
MYSQLDB_DATABASE=basicosd
MYSQLDB_LOCAL_PORT=3806
MYSQLDB_DOCKER_PORT=3306

POSGRE_DB=basicosdpostgres
POSGRES_USER=postuser
POSTGRES_PASSWORD=postpass
POSTGRES_PORT=5509
```

e. Código HTML

index.html

docker-compose.yml | pom.xml (basico) | application.properties | .env | index.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" xmlns:sec="http://www.thymeleaf.org/extras/spring-security">
<head>
    <meta charset="UTF-8">
    <title>Sistema de Gestión de Excepciones</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 50px; }
        .header { background-color: #f0f0f0; padding: 20px; border-radius: 5px; margin-bottom: 20px; }
        .menu { margin: 20px 0; }
        .menu a { display: inline-block; margin: 5px 10px 5px 0; padding: 10px 15px; background-color: #007bff; color: white; text-decoration: none; border-radius: 3px; }
        .logout-btn { background-color: #dc3545; border: none; padding: 10px 15px; color: white; cursor: pointer; border-radius: 3px; }
    </style>
</head>
<body>
<div class="header">
    <h1>P2: Sistema de Gestión de Excepciones</h1>
    <p>Demostración de manejo de errores con Spring Boot + Python Flask</p>
</div>

<div sec:authorize="isAnonymous()">
    <div class="menu">
        <a href="/login">Iniciar Sesión</a>
    </div>
</div>

<div sec:authorize="isAuthenticated()">
    <div class="menu">
        <p>Bienvenido, <strong sec:authentication="name"></strong>!
        <form th:action="@{/logout}" method="post" style="...">
            <button type="submit" class="logout-btn">Cerrar Sesión</button>
        </form>
        </p>
        <a href="/python-api">🔧 Test API Python (Simulación de Excepciones)</a>
    </div>


</div>
```

```html
<div sec:authorize="isAuthenticated()">
    <div class="menu">
        <p>Bienvenido, <strong sec:authentication="name"></strong>!
        <form th:action="@{/logout}" method="post" style="...">
            <button type="submit" class="logout-btn">Cerrar Sesión</button>
        </form>
        </p>
        <a href="/python-api">🔧 Test API Python (Simulación de Excepciones)</a>
    </div>


</div>

<div style="...">
    <h3>📌 Excepciones que se pueden testear:</h3>
    <ul>
        <li>❌ Error de Lectura de Archivos</li>
        <li>❌ Error de Acceso a Base de Datos</li>
        <li>❌ Error Llamando a API Externa (Pokémon)</li>
    </ul>
</div>
</body>
</html>
```

login.html

docker-compose.yml | pom.xml (basico) | application.properties | .env | index.html | login.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Sistema de Gestión de Excepciones - Login</title>
    <style>
        .error { color: red; }
        .container { width: 300px; margin: 100px auto; padding: 20px; border: 1px solid #ccc; border-radius: 5px; }
        input { width: 100%; padding: 8px; margin: 5px 0 15px 0; }
        button { width: 100%; padding: 10px; background-color: #007bff; color: white; border: none; cursor: pointer; }
    </style>
</head>
<body>
<div class="container">
    <h2>Iniciar Sesión</h2>
    <form th:action="@{/login}" method="post">
        <div th:if="${param.error}" class="error">
            <p>Usuario o Contraseña incorrectos</p>
        </div>
        <div th:if="${param.logout}" class="error">
            <p>Se ha Cerrado Sesión Correctamente!!!</p>
        </div>
        <div>
            <label>Usuario:</label>
            <input type="text" name="username" required/>
        </div>
        <div>
            <label>Contraseña:</label>
            <input type="password" name="password" required/>
        </div>
        <button type="submit">Entrar</button>
    </form>
    <p style="...">Usuarios: admin (admin123) | user (user123)</p>
</div>
</body>
</html>
```

error.html

docker-compose.yml | pom.xml (basico) | application.properties | .env | index.html | login.html | error.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Sistema de Gestión de Excepciones - Error</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 50px; }
        .error-container { border: 2px solid red; padding: 20px; border-radius: 5px; background-color: #fff3f3; }
        h2 { color: red; }
        pre { background-color: #f8f8f8; padding: 10px; overflow-x: auto; }
        a { color: #007bff; text-decoration: none; }
    </style>
</head>
<body>
<div class="error-container">
    <h2>⚠ Se ha producido una excepción</h2>
    <h3 th:text="${titulo}">Tipo de error</h3>
    <pre th:text="${error}">Detalles del error</pre>
    <a href="/">Volver a inicio</a>
</div>
</body>
</html>
```

python-api.html

docker-compose.yml | pom.xml (basico) | application.properties | .env | index.html | login.html | error.html | python-api.html

```html
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
  <meta charset="UTF-8">
  <title>Sistema de Gestión de Excepciones - Pruebas API Python</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 50px; }
    .container { max-width: 800px; margin: 0 auto; }
    .card { border: 1px solid #ddd; border-radius: 8px; padding: 20px; margin-bottom: 20px; }
    select, button { padding: 10px; margin: 10px 0; }
    button { background-color: #28a745; color: white; border: none; cursor: pointer; }
    button:hover { background-color: #218838; }
    .success { color: green; background-color: #d4edda; padding: 15px; border-radius: 5px; margin: 10px 0; }
    .error { color: red; background-color: #f8d7da; padding: 15px; border-radius: 5px; margin: 10px 0; }
    .info { color: blue; background-color: #d1ecf1; padding: 15px; border-radius: 5px; margin: 10px 0; }
    .back-link { display: inline-block; margin-top: 20px; color: #007bff; text-decoration: none; }
    .loading { color: orange; font-style: italic; }
    .error-details { background-color: #f5f5f5; padding: 15px; border-radius: 5px; font-family: monospace; font-size: 14px; overflow-x: auto; white-space: pre-wrap; word-wrap: b
  </style>
</head>
<body>
<div class="container">
  <h1>🔧 Simulador de Excepciones</h1>

  <div class="card">
    <form method="post" th:action="@{/python-api/test}">
      <select name="tipoError" required>
        <option value="">-- Selecciona el tipo de error a simular --</option>
        <option value="archivo">Error de Lectura de Archivo</option>
        <option value="bd">Error de Base de Datos</option>
        <option value="pokemon">Error Llamada a API externa (Pokemon)</option>
      </select>
      <button type="submit">Probar Excepción</button>
    </form>
  </div>
```

```html
    <div th:if="${loading}">
      <div class="info">Llamando al API Python, esperando respuesta...</div>
    </div>

    <!-- Información del tipo de error probado -->
    <div th:if="${tipoProbado} and !${loading}">
      <div class="info">
        <strong>ℹ️ Tipo de error probado:</strong>
        <span th:text="${tipoProbado}"></span>
      </div>
    </div>

    <!-- Error capturado -->
    <div th:if="${error} and !${loading}">
      <div class="error">
        <strong>❌ EXCEPCIÓN CAPTURADA:</strong>
        <div class="error-details" th:text="${error}"></div>
      </div>
    </div>
    <div style="...">
      <h3>📌 ¿Qué excepciones se están probando?</h3>
      <ul>
        <li><strong>Archivo:</strong> Intenta leer un archivo que no existe</li>
        <li><strong>Base de Datos:</strong> Consulta una tabla que no existe</li>
        <li><strong>Pokemon API:</strong> Busca un Pokémon con ID inválido (999999)</li>
      </ul>
      <p style="...">Todas las excepciones son capturadas por el API Python y devueltas como JSON con código 500.</p>
    </div>

    <a href="/" class="back-link">← Volver a inicio</a>
  </div>
  </body>
  </html>
```

f. app.py

docker-compose.yml | pom.xml (basico) | application.properties | .env | index.html | login.html | error.html | python-api.html | app.py

```python
from flask import Flask, jsonify
from flask_cors import CORS
import mysql.connector
import requests
import os

app = Flask(__name__)
CORS(app)

# Configuración de la base de datos (misma que usa Spring Boot)
db_config = {
    'host': 'localhost',
    'port': 3306,
    'user': 'root',
    'password': 'eneas2805',
    'database': 'basicosd'
}

@app.route('/api/health', methods=['GET'])
def health():
    return jsonify({"status": "OK", "message": "API Python funcionando"})
```

```python
@app.route('/api/test-exception/<tipo>', methods=['GET'])
def test_exception(tipo):
    try:
        if tipo == 'archivo':
            # Error: archivo no existe
            with open('/archivo_que_no_existe.txt', 'r') as f:
                contenido = f.read()
            return jsonify({"resultado": "Archivo leído", "contenido": contenido})

        elif tipo == 'bd':
            # Error: tabla no existe
            conn = mysql.connector.connect(**db_config)
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM tabla_que_no_existe")
            datos = cursor.fetchall()
            cursor.close()
            conn.close()
            return jsonify({"resultado": "Consulta exitosa", "datos": datos})

        elif tipo == 'pokemon':
            # Error: Pokémon inexistente
            url = "https://pokeapi.co/api/v2/pokemon/999999"
            response = requests.get(url, timeout=10)
            if response.status_code == 404:
                return jsonify({"error": "Pokémon no encontrado"}), 404
            return jsonify({"resultado": response.json()})

        else:
            return jsonify({"error": f"Tipo '{tipo}' no reconocido"}), 400
```

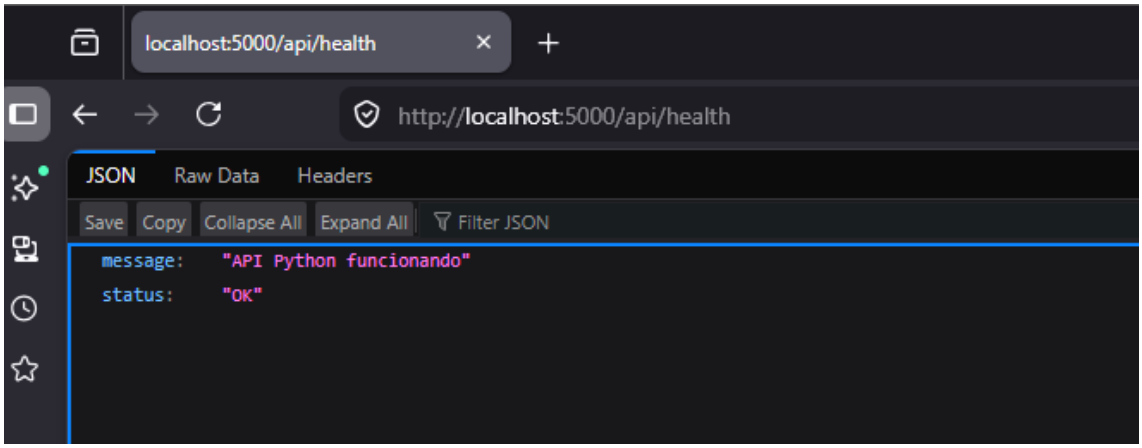
```python
    except FileNotFoundError as e:
        return jsonify({"error": f"Excepción de Archivo: {str(e)}"}), 500
    except mysql.connector.Error as e:
        return jsonify({"error": f"Excepción de BD: {str(e)}"}), 500
    except requests.exceptions.RequestException as e:
        return jsonify({"error": f"Excepción de API Externa: {str(e)}"}), 500
    except Exception as e:
        return jsonify({"error": f"Excepción genérica: {str(e)}"}), 500

if __name__ == '__main__':
    print("=== Iniciando API Python ===")
    print("Servidor en http://localhost:5000")
    app.run(debug=True, host='0.0.0.0', port=5000)
```

3. Prueba de Ejecución

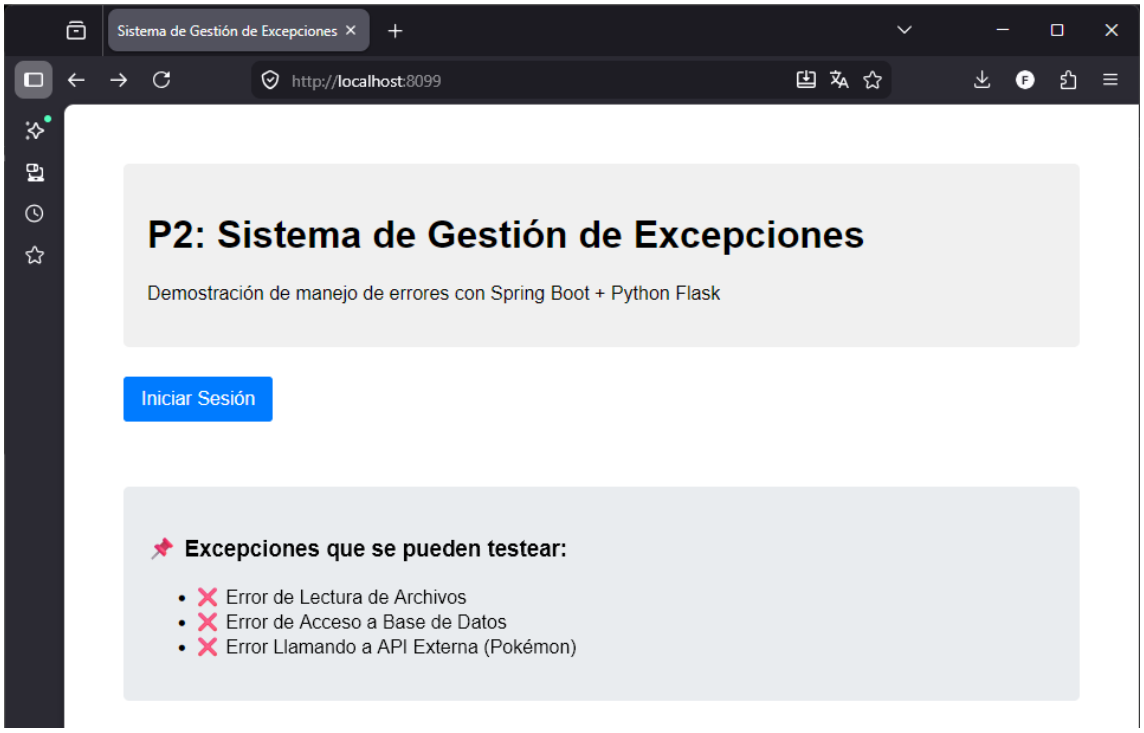
a. Capturas de los servidores en funcionamiento

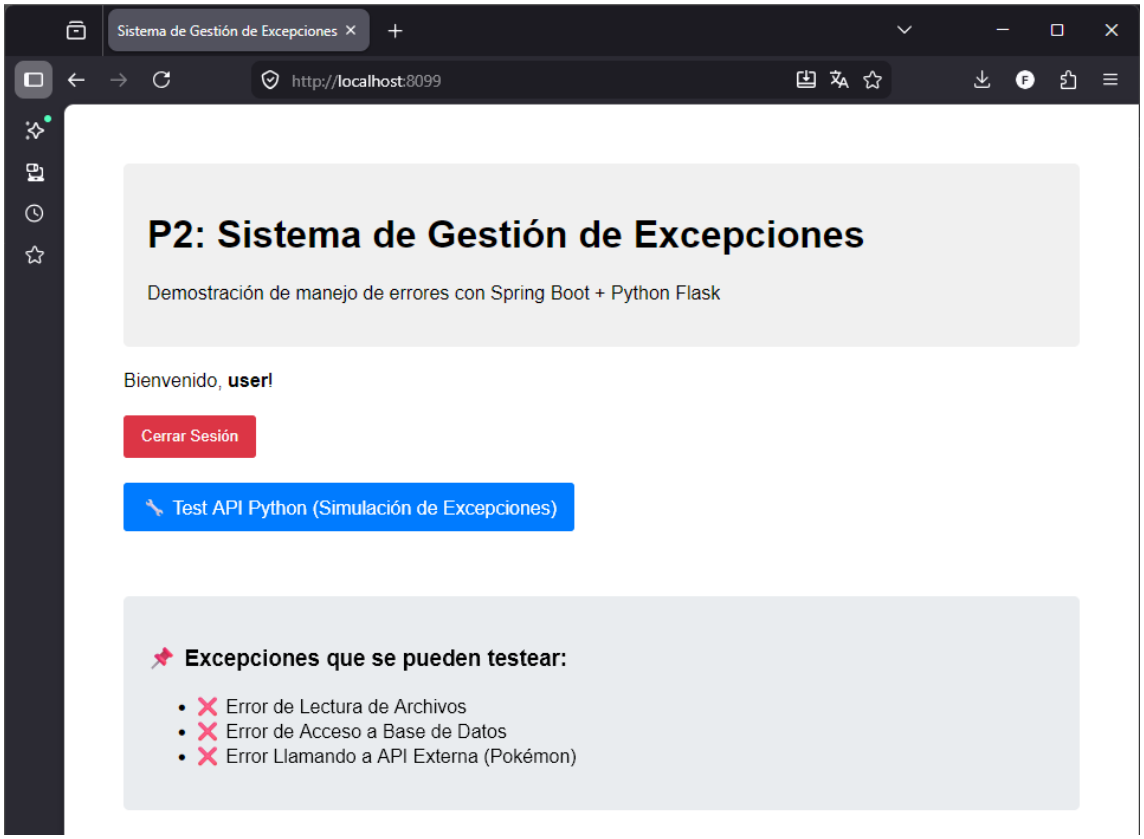
```
PS C:\SD_Programas\Proyectos\basico\api-python> python .\app.py
=== Iniciando API Python ===
Servidor en http://localhost:5000
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://192.168.8.153:5000
Press CTRL+C to quit
 * Restarting with stat
=== Iniciando API Python ===
Servidor en http://localhost:5000
 * Debugger is active!
 * Debugger PIN: 442-954-920
127.0.0.1 - - [21/Apr/2026 10:51:42] "GET / HTTP/1.1" 404 -
127.0.0.1 - - [21/Apr/2026 10:51:42] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [21/Apr/2026 10:52:16] "GET /api/health HTTP/1.1" 200 -
127.0.0.1 - - [21/Apr/2026 10:52:16] "GET /favicon.ico HTTP/1.1" 404 -
```



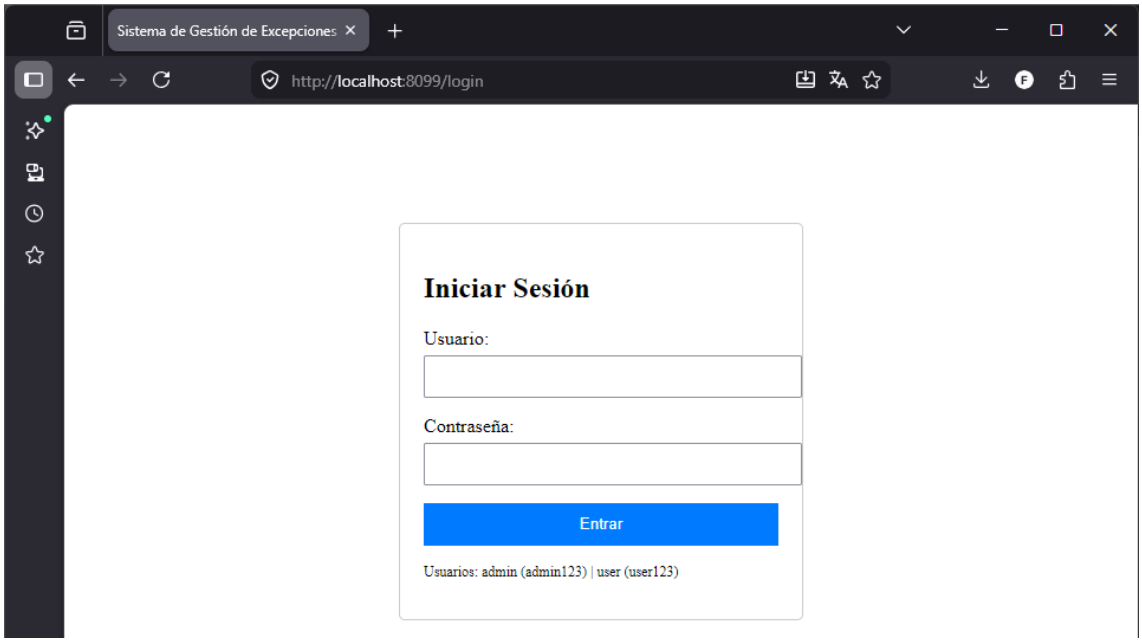

```
Windows PowerShell
Mapping      : Adding welcome page template: index
2026-04-21T12:53:59.840+02:00  INFO 6752 --- [basicosd] [           main] o.s.boot.tomcat.TomcatWebSer
ver          : Tomcat started on port 8099 (http) with context path '/'
2026-04-21T12:53:59.847+02:00  INFO 6752 --- [basicosd] [           main] c.s.basico.BasicoApplication
             : Started BasicoApplication in 6.118 seconds (process running for 6.563)
=== Inicializando datos ===
2026-04-21T12:54:00.161+02:00  INFO 6752 --- [basicosd] [nio-8099-exec-1] o.a.c.c.C.[Tomcat].[localhos
t].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-04-21T12:54:00.161+02:00  INFO 6752 --- [basicosd] [nio-8099-exec-1] o.s.web.servlet.DispatcherSe
rvlet        : Initializing Servlet 'dispatcherServlet'
2026-04-21T12:54:00.170+02:00  INFO 6752 --- [basicosd] [nio-8099-exec-1] o.s.web.servlet.DispatcherSe
rvlet        : Completed initialization in 9 ms
Hibernate: select count(*) from role r1_0
Creando roles...
Hibernate: insert into role (role_name,show_on_create) values (?,?)
Hibernate: insert into role (role_name,show_on_create) values (?,?)
Roles creados
Hibernate: select next_val as id_val from user_seq for update
Hibernate: update user_seq set next_val= ? where next_val=?
Hibernate: insert into user (email,fecha_ultimo_acceso,nombre_usuario,password,publickey,user_role_id,
username,id) values (?,?,?,?,?,?,?,?)
Hibernate: select next_val as id_val from user_seq for update
Hibernate: update user_seq set next_val= ? where next_val=?
Hibernate: insert into user (email,fecha_ultimo_acceso,nombre_usuario,password,publickey,user_role_id,
username,id) values (?,?,?,?,?,?,?,?)
Usuarios creados - admin/admin123, user/user123
=== Inicialización completa ===
```

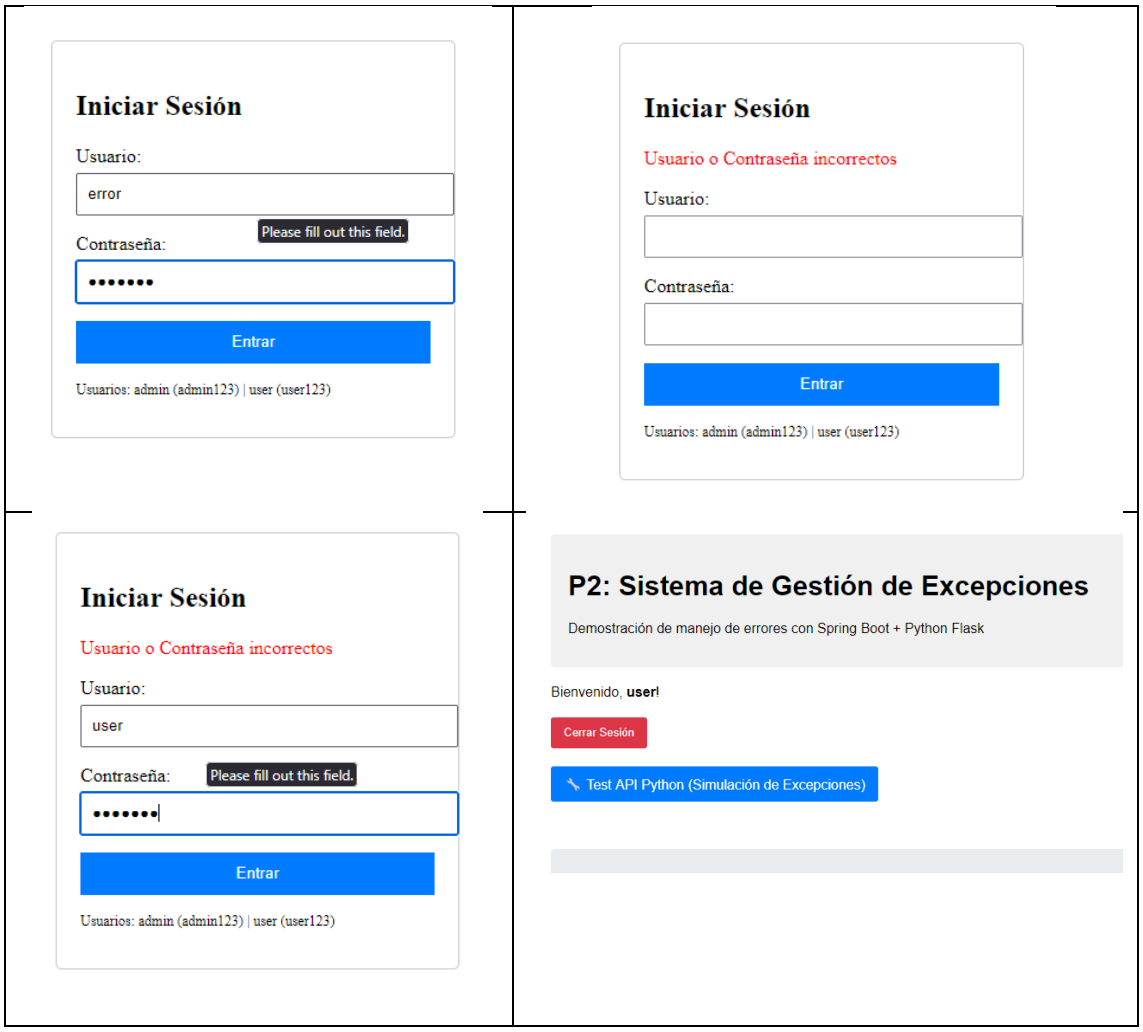
b. Página de Inicio



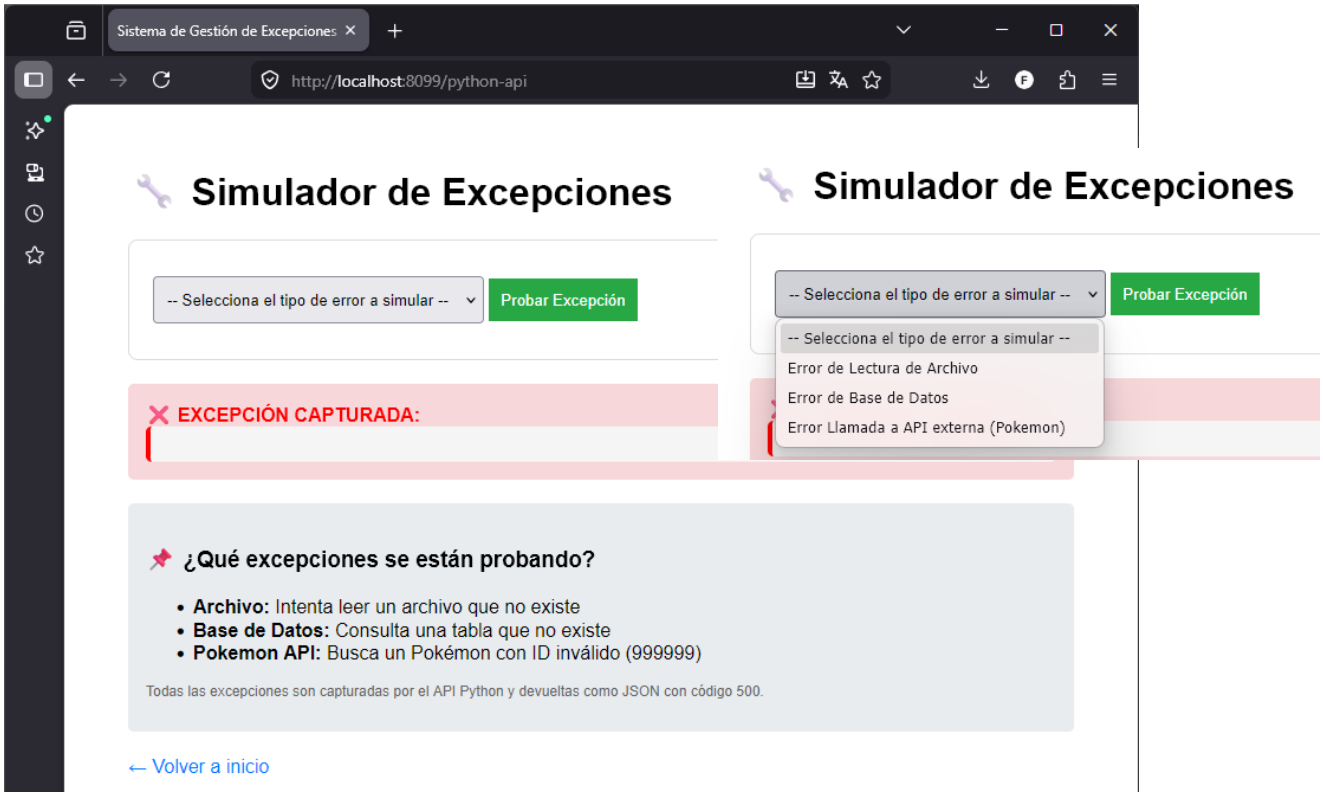


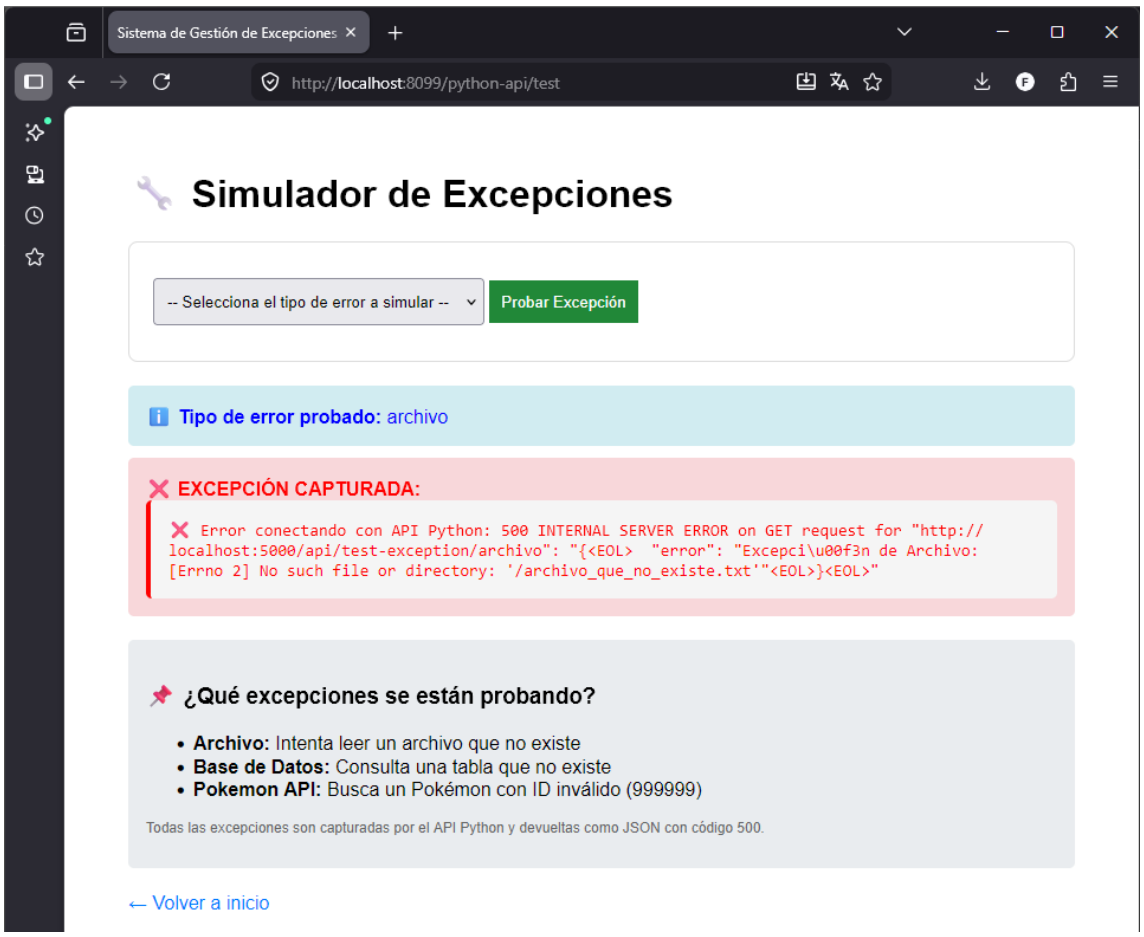
c. Página de Login





d. Página de Test





Sistema de Gestión de Excepciones

http://localhost:8099/python-api/test

Simulador de Excepciones

-- Selecciona el tipo de error a simular --

Probar Excepción

Tipo de error probado: bd

EXCEPCIÓN CAPTURADA:

```
Error conectando con API Python: 500 INTERNAL SERVER ERROR on GET request for "http://localhost:5000/api/test-exception/bd": "{<EOL>  "error": "Excepci\u00f3n de BD: 1146 (42S02): Table 'basicosd.tabla_que_no_existe' doesn't exist"<EOL>}<EOL>"
```

¿Qué excepciones se están probando?

- **Archivo:** Intenta leer un archivo que no existe
- **Base de Datos:** Consulta una tabla que no existe
- **Pokemon API:** Busca un Pokémon con ID inválido (999999)

Todas las excepciones son capturadas por el API Python y devueltas como JSON con código 500.

← Volver a inicio

Sistema de Gestión de Excepciones

http://localhost:8099/python-api/test

Simulador de Excepciones

-- Selecciona el tipo de error a simular --

Probar Excepción

Tipo de error probado: pokemon

EXCEPCIÓN CAPTURADA:

```
Error conectando con API Python: 404 NOT FOUND on GET request for "http://localhost:5000/api/test-exception/pokemon": "{<EOL>  "error": "Pok\u00e9mon no encontrado"<EOL>}<EOL>"
```

¿Qué excepciones se están probando?

- **Archivo:** Intenta leer un archivo que no existe
- **Base de Datos:** Consulta una tabla que no existe
- **Pokemon API:** Busca un Pokémon con ID inválido (999999)

Todas las excepciones son capturadas por el API Python y devueltas como JSON con código 500.

← Volver a inicio

4. Estado de las operaciones en los servidores

```
Windows PowerShell
 * Debugger is active!
 * Debugger PIN: 442-954-920
127.0.0.1 - - [21/Apr/2026 12:47:50] "GET /api/health HTTP/1.1" 200 -
127.0.0.1 - - [21/Apr/2026 12:47:50] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [21/Apr/2026 12:47:55] "GET /api/test-exception/bd HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:48:04] "GET /api/test-exception/archivo HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:48:07] "GET /api/test-exception/bd HTTP/1.1" 500 -
 * Detected change in 'C:\\SD_Programas\\Proyectos\\basico\\api-python\\app.py', reloading
 * Restarting with stat
=== Iniciando API Python ===
Servidor en http://localhost:5000
 * Debugger is active!
 * Debugger PIN: 442-954-920
PS C:\SD_Programas\Proyectos\basico\api-python> python .\app.py
=== Iniciando API Python ===
Servidor en http://localhost:5000
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
 server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.83.58.20:5000
Press CTRL+C to quit
 * Restarting with stat
=== Iniciando API Python ===
Servidor en http://localhost:5000
 * Debugger is active!
 * Debugger PIN: 442-954-920
127.0.0.1 - - [21/Apr/2026 12:50:52] "GET /api/test-exception/archivo HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:50:56] "GET /api/test-exception/bd HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:51:09] "GET /api/test-exception/pokemon HTTP/1.1" 404 -
127.0.0.1 - - [21/Apr/2026 12:51:28] "GET /api/test-exception/archivo HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:51:31] "GET /api/test-exception/bd HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:51:34] "GET /api/test-exception/pokemon HTTP/1.1" 404 -
127.0.0.1 - - [21/Apr/2026 12:51:39] "GET /api/test-exception/archivo HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:51:42] "GET /api/test-exception/pokemon HTTP/1.1" 404 -
127.0.0.1 - - [21/Apr/2026 12:54:14] "GET /api/test-exception/archivo HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:55:37] "GET /api/test-exception/archivo HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:55:44] "GET /api/test-exception/bd HTTP/1.1" 500 -
127.0.0.1 - - [21/Apr/2026 12:55:52] "GET /api/test-exception/pokemon HTTP/1.1" 404 -
```

```
=== Inicialización completa ===
Hibernate: select u1_0.id,u1_0.email,u1_0.fecha_ultimo_acceso,u1_0.nombre_usuario,u1_0.password,u1_0.p
ublickey,u1_0.user_role_id,u1_0.username from user u1_0 where u1_0.username=?
Hibernate: select r1_0.id,r1_0.role_name,r1_0.show_on_create from role r1_0 where r1_0.id=?
Llamando a Python API: http://localhost:5000/api/test-exception/archivo
Error al llamar Python API: 500 INTERNAL SERVER ERROR on GET request for "http://localhost:5000/api/te
st-exception/archivo": "{<EOL>  "error": "Excepci\u00f3n de Archivo: [Errno 2] No such file or directo
ry: '/archivo_que_no_existe.txt'"<EOL>}<EOL>"
Hibernate: select u1_0.id,u1_0.email,u1_0.fecha_ultimo_acceso,u1_0.nombre_usuario,u1_0.password,u1_0.p
ublickey,u1_0.user_role_id,u1_0.username from user u1_0 where u1_0.username=?
Hibernate: select u1_0.id,u1_0.email,u1_0.fecha_ultimo_acceso,u1_0.nombre_usuario,u1_0.password,u1_0.p
ublickey,u1_0.user_role_id,u1_0.username from user u1_0 where u1_0.username=?
Hibernate: select r1_0.id,r1_0.role_name,r1_0.show_on_create from role r1_0 where r1_0.id=?
Llamando a Python API: http://localhost:5000/api/test-exception/archivo
Error al llamar Python API: 500 INTERNAL SERVER ERROR on GET request for "http://localhost:5000/api/te
st-exception/archivo": "{<EOL>  "error": "Excepci\u00f3n de Archivo: [Errno 2] No such file or directo
ry: '/archivo_que_no_existe.txt'"<EOL>}<EOL>"
Llamando a Python API: http://localhost:5000/api/test-exception/bd
Error al llamar Python API: 500 INTERNAL SERVER ERROR on GET request for "http://localhost:5000/api/te
st-exception/bd": "{<EOL>  "error": "Excepci\u00f3n de BD: 1146 (42S02): Table 'basicosd.tabla_que_no_
existe' doesn't exist"<EOL>}<EOL>"
Llamando a Python API: http://localhost:5000/api/test-exception/pokemon
Error al llamar Python API: 404 NOT FOUND on GET request for "http://localhost:5000/api/test-exception
/pokemon": "{<EOL>  "error": "Pok\u00e9mon no encontrado"<EOL>}<EOL>"
```